

Phase 1 Report - High-Performance Linear Algebra Kernels

Team

- Names: Jonathan Kim, Andrew Moukabary, Cole Koryto

This report summarizes the baseline implementations, benchmarking methodology, optimization, alignment tests, and profiling results for matrix-vector and matrix-matrix kernels for our Phase 1 report.

Methods

- Four baseline kernels: MV row-major, MV column-major, MM naive, MM with transposed B.
- One optimized MM variant using blocking/tiling.
- Benchmarks: square matrices ($N = 64, 128, 256, 512, 1024$), 20 runs per size, `std::chrono` timing around the kernel only.
- Profiling: `./main profile` on 1024×1024 for MM kernels.

Implementations

- MV Row-Major:** Iterates rows of A and columns of the vector. Inner loop walks contiguous memory for each row.
- MV Column-Major:** Uses column-major indexing; inner loop accesses strided memory across columns.
- MM Naive:** Triple loop with row-major A and B; result is written in row-major order.
- MM Transposed B:** Multiplies A by pre-transposed B to improve access locality in the inner loop.
- MM Optimized (Blocked):** Applies blocking/tiling to improve cache reuse across submatrices.

Correctness

- Deterministic tests validate MV and MM kernels on small square and rectangular cases, and the expected outputs are compared against computed results with a tight numeric tolerance.

Benchmark Results (With -O3)

Size	Function	Avg (ms)	Stddev (ms)
64	mv_row_major	0.0020334	0.000871734
64	mv_col_major	0.00443545	0.00181468
64	mm_naive	0.351846	0.438274

Size	Function	Avg (ms)	Stddev (ms)
64	mm_transposed_b	0.126742	0.0730503
64	mm_optimized	0.0693792	0.0186308
128	mv_row_major	0.0056562	9.21638e-05
128	mv_col_major	0.0121938	0.000110614
128	mm_naive	1.59885	0.00999228
128	mm_transposed_b	0.720085	0.00271691
128	mm_optimized	0.405856	0.00347975
256	mv_row_major	0.0305876	0.000825161
256	mv_col_major	0.0695124	0.00335166
256	mm_naive	18.3181	0.126508
256	mm_transposed_b	9.17798	0.0392309
256	mm_optimized	3.67675	0.024557
512	mv_row_major	0.166764	0.0631289
512	mv_col_major	0.419242	0.254654
512	mm_naive	184.089	12.5605
512	mm_transposed_b	77.3535	1.23002
512	mm_optimized	34.7869	2.64299
1024	mv_row_major	0.720327	0.00244212
1024	mv_col_major	1.44778	0.0321276
1024	mm_naive	2035.49	9.80046
1024	mm_transposed_b	740.791	35.2292
1024	mm_optimized	309.573	0.352854

Benchmark Results (Without Optimization, -O0)

Size	Function	Avg (ms)	Stddev (ms)
64	mv_row_major	0.0092209	0.00367494
64	mv_col_major	0.00818535	0.00318406
64	mm_naive	0.629352	0.1804
64	mm_transposed_b	0.65496	0.163748
64	mm_optimized	0.496152	0.127236
128	mv_row_major	0.0333105	0.0119082
128	mv_col_major	0.0390626	0.00958077
128	mm_naive	6.32702	0.269928
128	mm_transposed_b	4.69785	0.0721536
128	mm_optimized	3.6204	0.0317699
256	mv_row_major	0.136254	0.0571128
256	mv_col_major	0.257331	0.0101137
256	mm_naive	68.0564	0.657049
256	mm_transposed_b	37.9747	0.197806
256	mm_optimized	28.9425	0.111139
512	mv_row_major	0.523552	0.171493

Size	Function	Avg (ms)	Stddev (ms)
512	mv_col_major	1.14502	0.0223036
512	mm_naive	602.7	20.125
512	mm_transposed_b	312.735	0.888471
512	mm_optimized	235.64	0.849742
1024	mv_row_major	1.91212	0.632103
1024	mv_col_major	4.70922	0.0337035
1024	mm_naive	4937.19	81.0739
1024	mm_transposed_b	2445.85	13.0341
1024	mm_optimized	1908.61	4.06285

Alignment Benchmark

Aligned vs unaligned timings were measured for MM kernels (64-byte alignment). Results are shown below.

Size	Function	Allocation	Avg (ms)	Stddev (ms)
64	mm_naive	unaligned	0.22319	0.0860065
64	mm_naive	aligned	0.220242	0.0913713
64	mm_transposed_b	unaligned	0.100569	0.0509846
64	mm_transposed_b	aligned	0.0988667	0.0473508
128	mm_naive	unaligned	1.58965	0.00318377
128	mm_naive	aligned	1.59033	0.00480297
128	mm_transposed_b	unaligned	0.721065	0.00393897
128	mm_transposed_b	aligned	0.72035	0.00363945
256	mm_naive	unaligned	16.3105	0.0707455
256	mm_naive	aligned	16.3295	0.099548
256	mm_transposed_b	unaligned	7.71237	0.0507495
256	mm_transposed_b	aligned	7.70068	0.0263582
512	mm_naive	unaligned	180.623	6.63196
512	mm_naive	aligned	186.508	34.0138
512	mm_transposed_b	unaligned	77.4751	1.50756
512	mm_transposed_b	aligned	77.4916	1.36434
1024	mm_naive	unaligned	2072.06	61.3784
1024	mm_naive	aligned	2077.61	46.2924
1024	mm_transposed_b	unaligned	735.4	4.0243
1024	mm_transposed_b	aligned	735.025	3.44498

Profiling

- Profiling shows that most runtime is spent inside the MM kernels' inner loops, consistent with the $O(N^3)$ cost. The naive MM highlights time in the innermost accumulation over the shared dimension, while the

transposed/blocked variants shift cost toward tighter, more cache-friendly loops.

Optimization Summary: Blocking/Tiling

- We implemented blocking and tiling for matrix multiplication, as we can reuse the A and B submatrices we create while simultaneously accumulating a block of C
 - This increases temporal locality and reduces cache misses
 - When N gets larger, our benefits increase as well: working sets can exceed cache capacity!
- Outcome: At N=1024 with -O3, MM naive averages 2035.49 ms while MM optimized averages 309.573 ms ($\sim 6.6\times$ faster). With -O0, MM optimized still improves over naive (4937.19 ms $\rightarrow$ 1908.61 ms).

Cache Locality Analysis

- **Row-major MV** benefits from contiguous row access; **column-major MV** incurs strided access, leading to more cache misses.
- **MM naive** repeatedly streams through B with poor reuse; **MM transposed B** and **MM blocked** improve spatial and temporal locality in the inner loops.
- The measured timings reflect these access patterns, with transposed and blocked variants consistently faster at larger sizes.

Alignment Analysis

- Aligned allocation showed minimal differences at most sizes for both MM naive and MM transposed B.
- Variance is within standard deviations for many sizes, suggesting alignment alone is not the dominant factor compared to access patterns and blocking.

Optimization Level Effects

- -O3 significantly reduces runtime for all kernels compared to -O0, enabling inlining, vectorization, and other loop optimizations.
- The relative speedup of the blocked MM is larger under -O3, indicating that compiler optimizations amplify cache-friendly loop structures.

Key Findings

- Row-major MV consistently outperforms column-major MV due to contiguous access patterns.
- Transposed and blocked MM variants improve cache reuse and reduce runtime substantially versus naive MM.

- Compiler optimizations provide large gains, especially for compute-heavy MM kernels.

Limitations and Notes

- Benchmarks exclude allocation and initialization time to isolate kernel performance.
- Results are obviously hardware-dependent; absolute timings will vary across machines.
- Further tuning (e.g., block size, loop ordering) may yield additional gains.